Hash Table with Separate Chaining vs Binary Search Tree

A dictionary stores **key–value pairs** and supports operations such as **insert, search, and delete**. In a multi-threaded environment, the main goal is to allow many threads to access the dictionary efficiently while avoiding data races.

Factor	Hash Table – Separate Chaining	Binary Search Tree (BST)
Basic structure	Array of buckets; collisions stored in linked lists	Hierarchical tree of nodes
Average search	**O(1)**	**O(log n)** if balanced
Worst-case search	O(n)	O(n) for an unbalanced BST
Locking granularity	Can use **one lock per bucket**	Can use node/subtree locks
Concurrent access	Different buckets can be accessed simultaneously	Different subtrees can be accessed simultaneously
Insertion	Simple; insert into appropriate bucket	Requires tree traversal and pointer updates
Deletion	Relatively simple	More complex due to node restructuring
Resizing	Expensive; may require rehashing many entries	No global resizing required

Factor	Hash Table – Separate Chaining	Binary Search Tree (BST)
Cache behavior	Good array locality, but chains cause pointer chasing	Usually poorer due to scattered tree nodes
Ordered traversal	Not naturally ordered	Naturally supports sorted-order traversal
Concurrency scalability	**Very high** with bucket-level locking	Moderate; depends on tree balancing and locking
Memory overhead	Buckets + chain nodes	Tree nodes + pointers

Locking Granularity

Separate chaining hash table:

- The table can be divided into multiple buckets.
- Each bucket can have its own lock.
- Two threads accessing different buckets can operate simultaneously.
- This greatly reduces lock contention.

BST:

- A simple implementation may use one global tree lock, which limits concurrency.

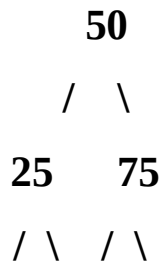
- More advanced implementations can use fine-grained node or sub tree locks.
- However, rotations and structural modifications may require multiple locks.

Therefore, fine-grained locking is generally easier to exploit effectively in a hash table.

Concurrent Access Patterns

If keys are well distributed, threads usually access different buckets. This creates high parallelism.

A BST has a hierarchical access pattern:



Resizing Complexity

When the load factor becomes too high, the table may need a larger bucket array and entries must be rehashed.

Old Table
[0][1][2][3]

↓ resize

New Table
[0][1][2][3][4][5][6][7]

A BST does not need table-wide resizing. Nodes are dynamically inserted and removed. However, a balanced BST may require rotations or rebalancing operations.

Cache Behavior

Hash tables generally have better cache behavior because the bucket array is contiguous in memory. However, separate chaining uses linked nodes, so following long chains causes **pointer chasing** and cache misses.

BSTs typically have poorer cache locality because nodes can be scattered throughout memory. Traversing from the root to a leaf involves following multiple pointers.

Thus:

Hash table → generally better cache locality

BST → generally poorer cache locality

Which Supports More Scalable Concurrent Operations?

A hash table with separate chaining generally supports more scalable concurrent dictionary operations, especially when:

- The hash function distributes keys evenly.
- Each bucket has an independent lock.

- The table is resized using an efficient concurrent resizing strategy.
- Most operations are independent searches/inserts on different keys.

Its average **O(1)** lookup and bucket-level locking allow many threads to operate simultaneously with relatively low contention.

A **BST** is preferable when the dictionary requires operations such as:

- Sorted traversal
- Finding minimum/maximum
- Range queries
- Predecessor/successor operations

Conclusion

For a **multi-threaded dictionary focused on fast search, insertion, and deletion**, a **separate-chaining hash table is usually more scalable** than a BST. Its bucket-level locking enables greater parallelism and its average O(1) operations provide better performance. However, if **ordered data and range-based operations** are important, a balanced concurrent BST may be the better choice despite its additional synchronization complexity.